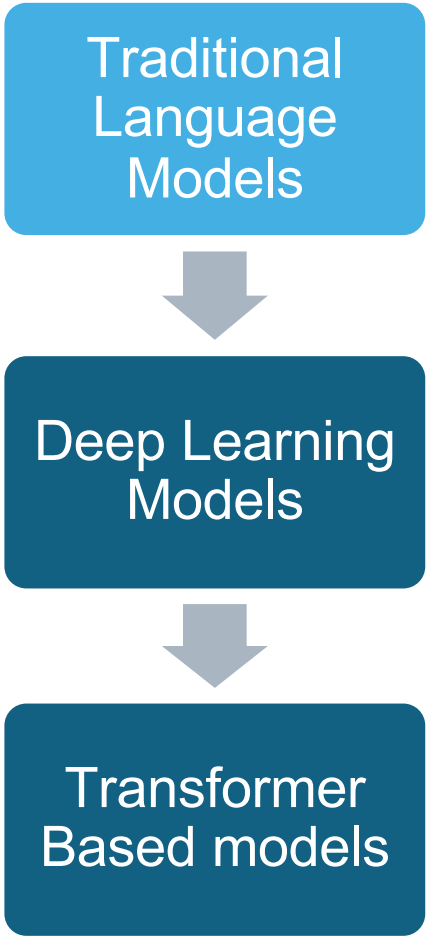
Large Language Models

Introduction to LLMs

Traditional Language Models:

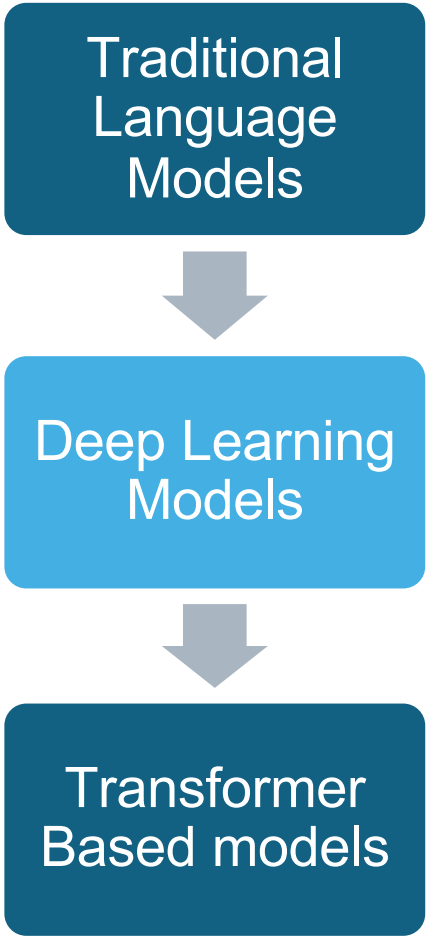
- **Limitations**: N-grams and Hidden Markov Models (HMMs) struggle with long-range dependencies because they only consider a fixed context window (e.g., only the last few words).

- **Markov Assumption**: These models assume that the probability of a word depends only on a limited number of preceding words (which limits their ability to capture broader context).



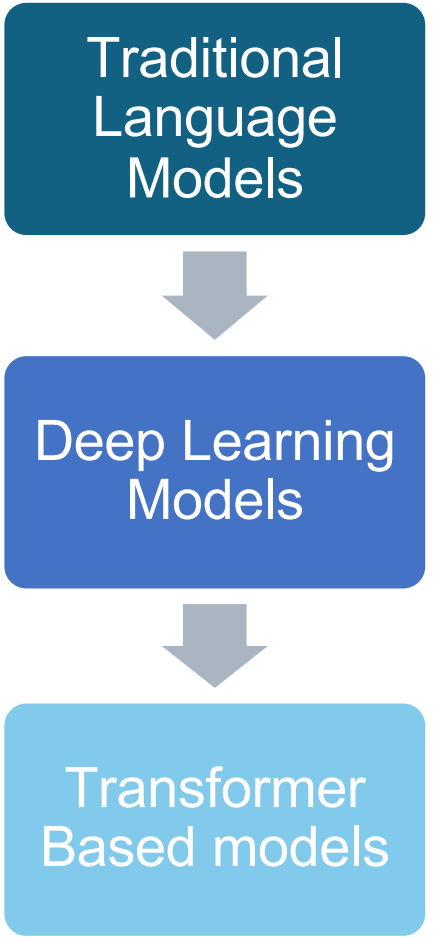
Deep Learning Language Models:

- **Recurrent Neural Networks (RNNs)**: RNNs improved over traditional models by introducing the ability to handle sequential data and maintain context over time.
    - **Limitations of RNNs**: Struggles with vanishing/exploding gradients, making it difficult to retain information over long sequences.
    - **Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU)**: Variants of RNNs designed to mitigate some of these issues by using gating mechanisms to better retain and forget information across time steps.



Transition to Transformers

- Transformer models made several improvements
  - **Parallel Processing**: By processing entire sequences at once, transformers significantly speed up training and inference compared to sequential models like RNNs.
  - **Self-Attention**: This mechanism allows transformers to focus on important tokens throughout the sequence, capturing long-range dependencies more effectively.
  - **Positional Encoding**: Since transformers handle all tokens simultaneously, positional encoding is used to maintain the order of words, ensuring the model understands word relationships.



What are Large Language Models

- LLMs are transformer-based language models trained on massive datasets.
- LLMs can be encoder-based (for **understanding** tasks), decoder-based (for **generative** tasks), or a combination of both (for tasks like **translation**).
- Generative LLMs, which are typically decoder-based, use the ability of decoders to generate new content based on the input they receive or the context they have processed.

Encoders:

- Process the entire input sequence at once.
- Useful for tasks requiring deep understanding and contextualization of the input (e.g., BERT).

Decoders:

- Generate text sequentially by predicting one token at a time.
- Key for generative tasks, such as translation or text completion (e.g., GPT).

Transformer

Encoder (Left Side):

- Consists of multiple layers with Multi-Head Attention and Feed Forward Networks
- Uses Add & Norm for stability
- Incorporates Positional Encoding to maintain word order

Decoder (Right Side):

- Similar structure with additional Masked Multi-Head Attention
- Attends to encoder output for context
- Generates outputs sequentially

Final Output:

- Linear transformation followed by Softmax to predict output probabilities

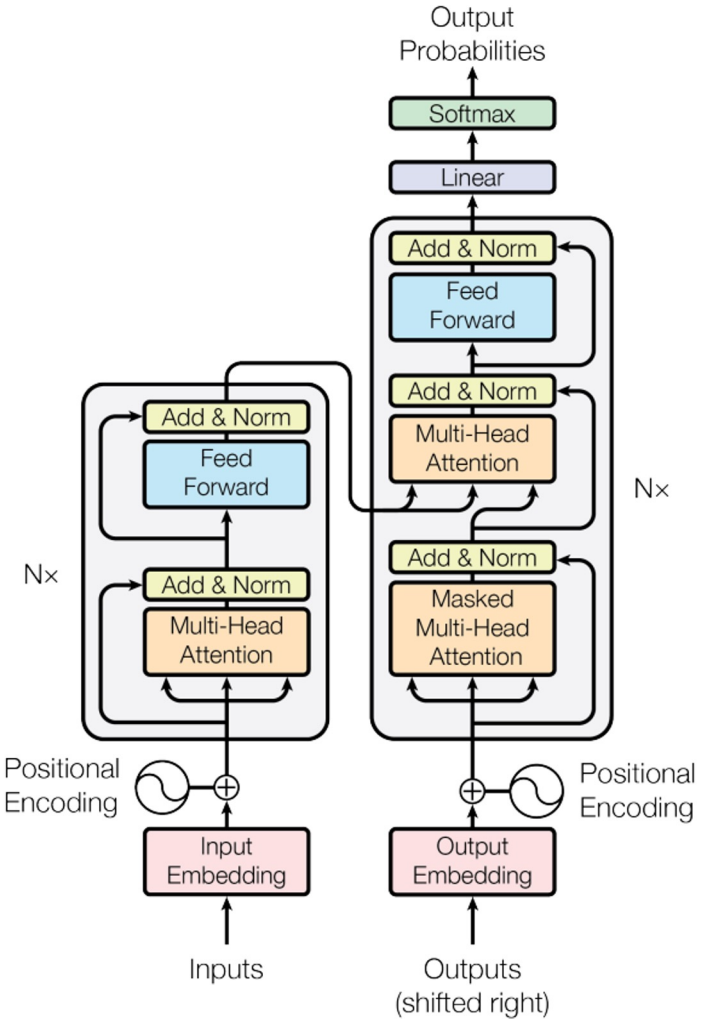


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformer

Encoder (Left Side):

- Consists of multiple layers with Multi-Head Attention and Feed Forward Networks
- Uses Add & Norm for stability
- Incorporates Positional Encoding to maintain word order

Decoder (Right Side):

- Similar structure with additional Masked Multi-Head Attention
- Attends to encoder output for context
- Generates outputs sequentially

Final Output:

- Linear transformation followed by Softmax to predict output probabilities

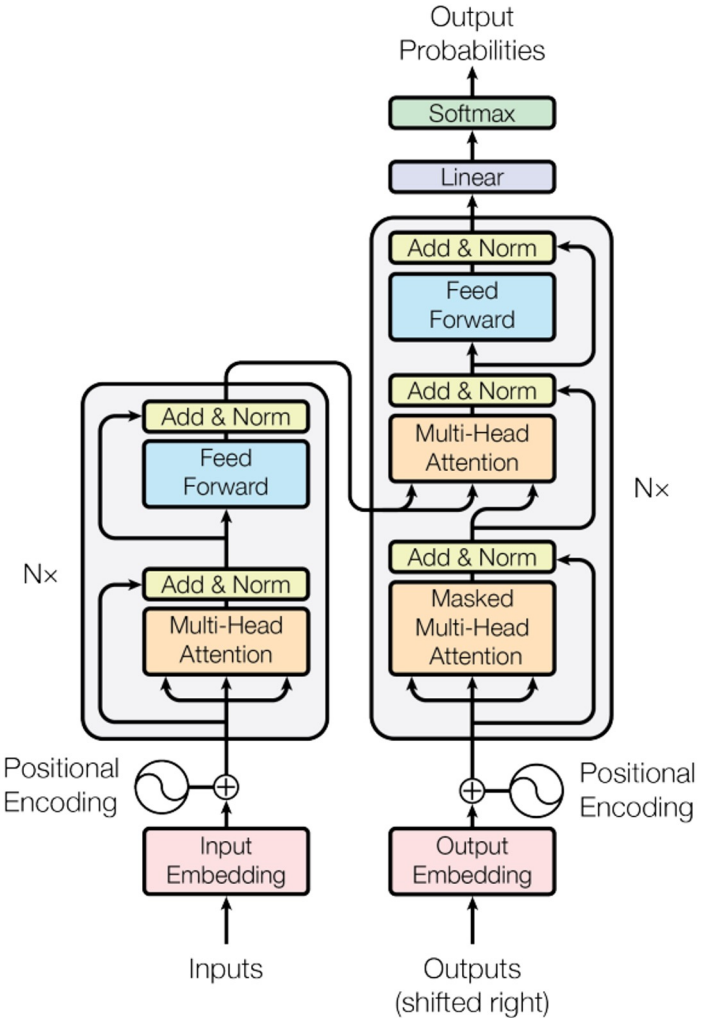


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformer - Encoder

- The transformer encoder processes the input sequence, transforming it into a contextualized representation through multiple layers of multi-head self-attention and feed-forward networks.

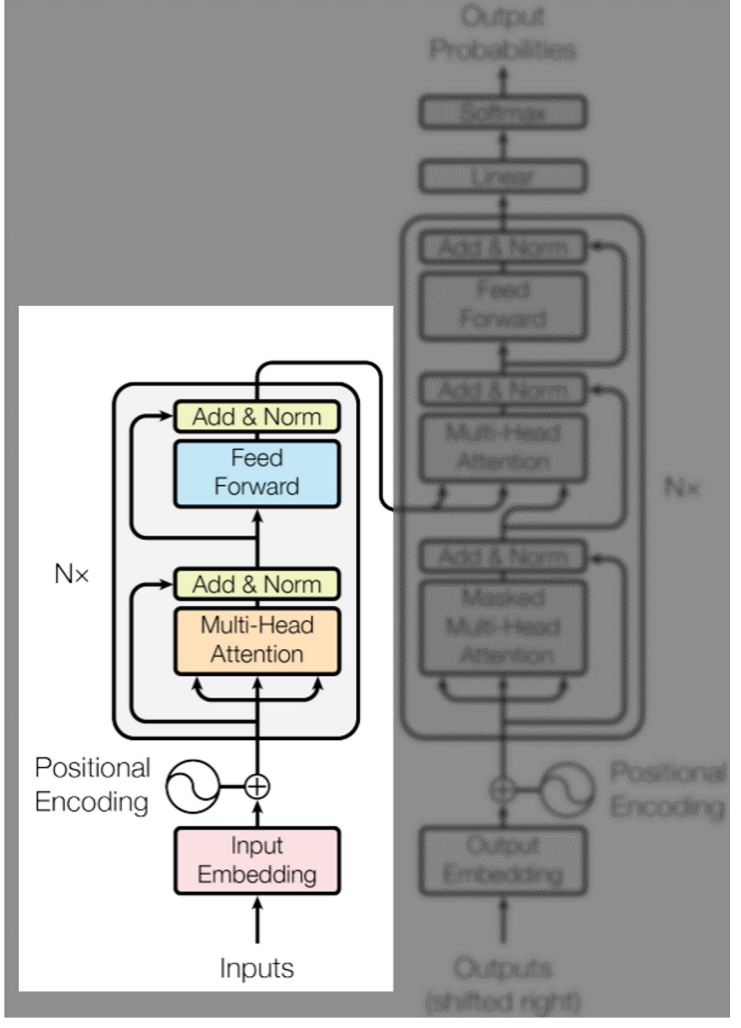


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformer - Decoder

- The transformer decoder generates output sequences by attending to both the input (from the encoder) and previously generated tokens.

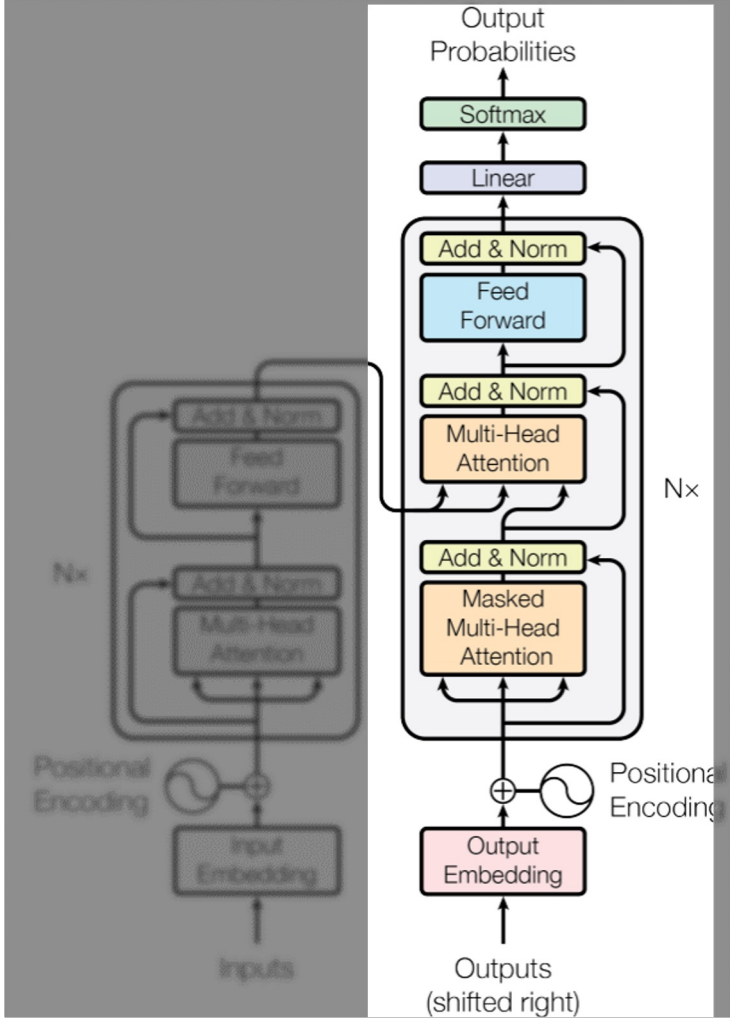


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Positional encoding

- In transformer models, **positional encoding** is crucial because transformers do not have an inherent sense of the order of tokens in a sequence.

- **Positional encoding** is a technique used to introduce information about the position of tokens in a sequence.

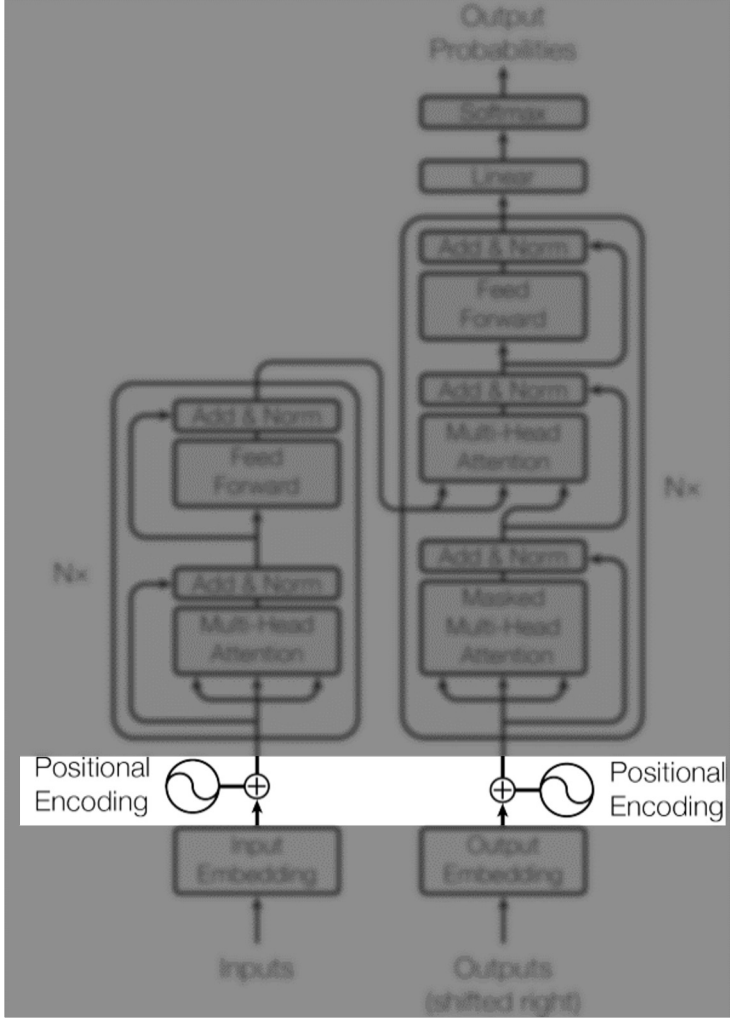


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

What does any of that mean?!

- Let's start by following a sentence as it travels through the transformer on the left.

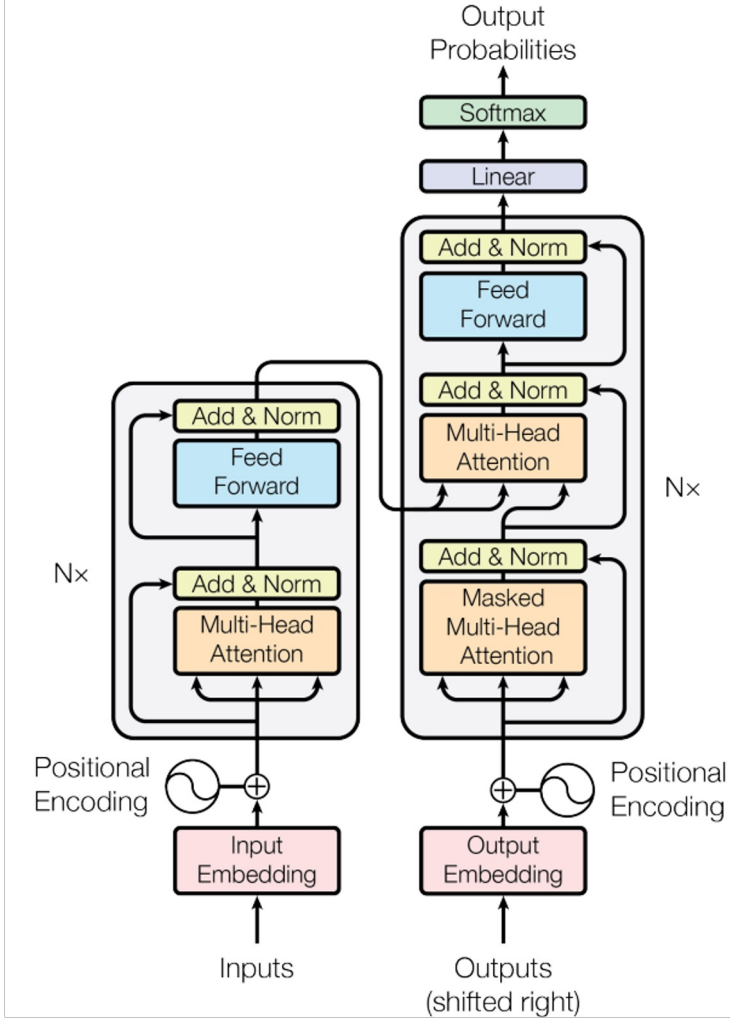


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Input

- Let's start by following a sentence as it travels through a standard transformer.
- Suppose you have a sentence.
- I am not a purple balloon.
- Simple tokenization
  - [‘i’, ‘am’, ’not’, ‘a’, ‘purple’ , ‘ballon’]

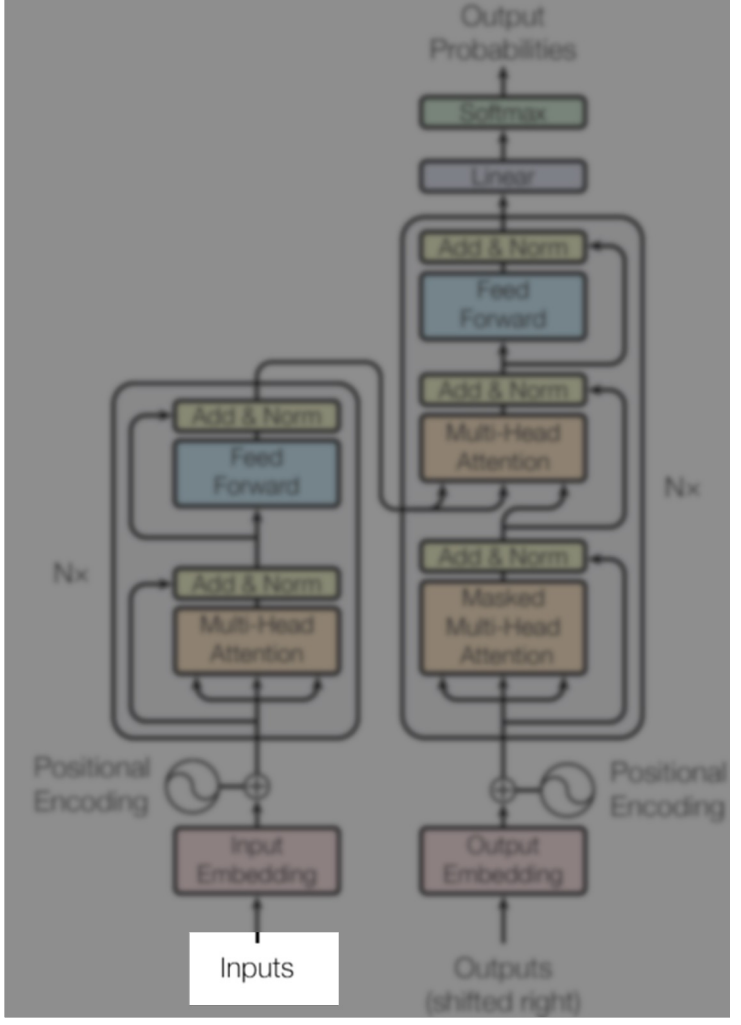


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Embedding Layer

- Standard embedding process
- [‘i’, ‘am’, ’not’, ‘a’, ‘purple’ , ‘ballon’]
- The output results in each word being associated with an embedding vector.

Word	Vector
‘i’	[0.1, 0.05, 0.2, ..., 0.4]
'am'	[0.3, -0.1, 0.2, ..., 0.7]
...	
'balloon'	[0.4, 0.1, 0.6, ..., -0.3]

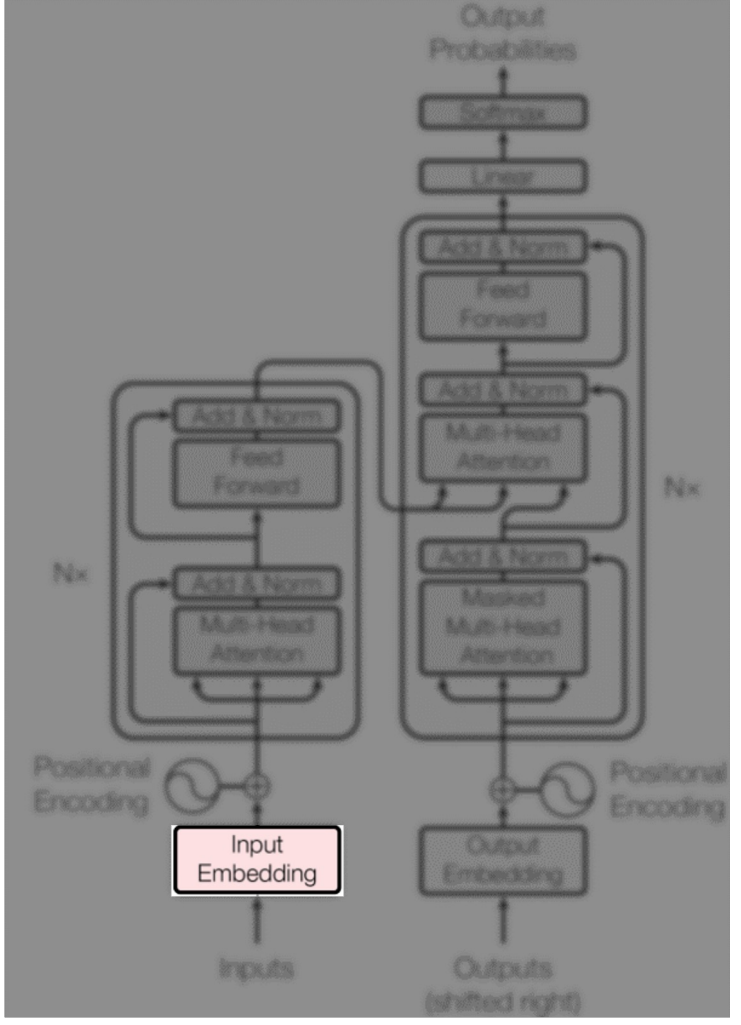


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Positional encoding

- **Why?**
  - Since transformers don't inherently know the position of tokens in a sequence, **positional encoding** is added to each word embedding to help the model understand the **order** of the words.

Position	Positional Encoding
0	[0.1, 0.05, 0.2, ..., 0.4]
1	[0.01, 0.02, -0.01, 0.03]
2	...
5	[0.02, -0.01, 0.03, 0.01]

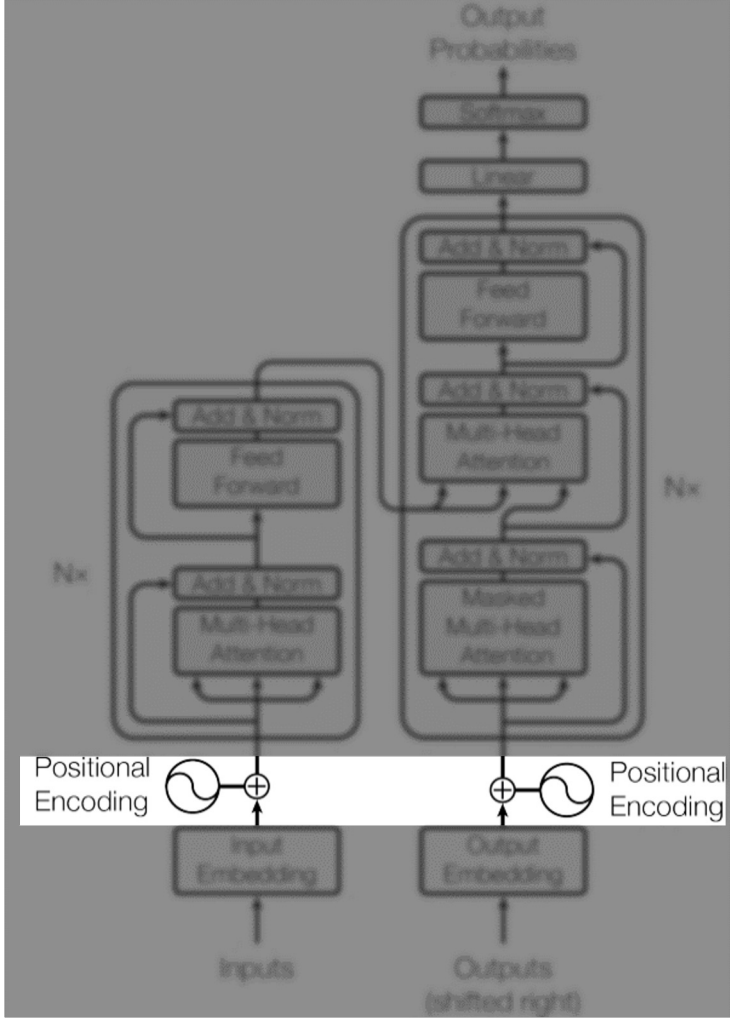


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Positional encoding + Embedding

- **Final Embedding:**
  - For each word, the positional encoding is added to the original word embedding.

Word	Vector
‘i’	[0.1, 0.05, 0.2, ..., 0.4]
'am'	[0.3, -0.1, 0.2, ..., 0.7]
...	
'balloon'	[0.4, 0.1, 0.6, ..., -0.3]

+

Position	Positional Encoding
0	[0.1, 0.06, 0.3, ..., 0.8]
1	[0.01, 0.02, -0.01, 0.03]
2	...
5	[0.02, -0.01, 0.03, 0.01]

  - This gives the model both the meaning of the word and its position in the sentence.

=

Word	Final embedding
i	[0.1 + 0.1, 0.05 + 0.06, 0.2 + 0.3, ..., 0.4 + 0.8]
i	[0.2, 0.11, 0.5, ..., 1.2]

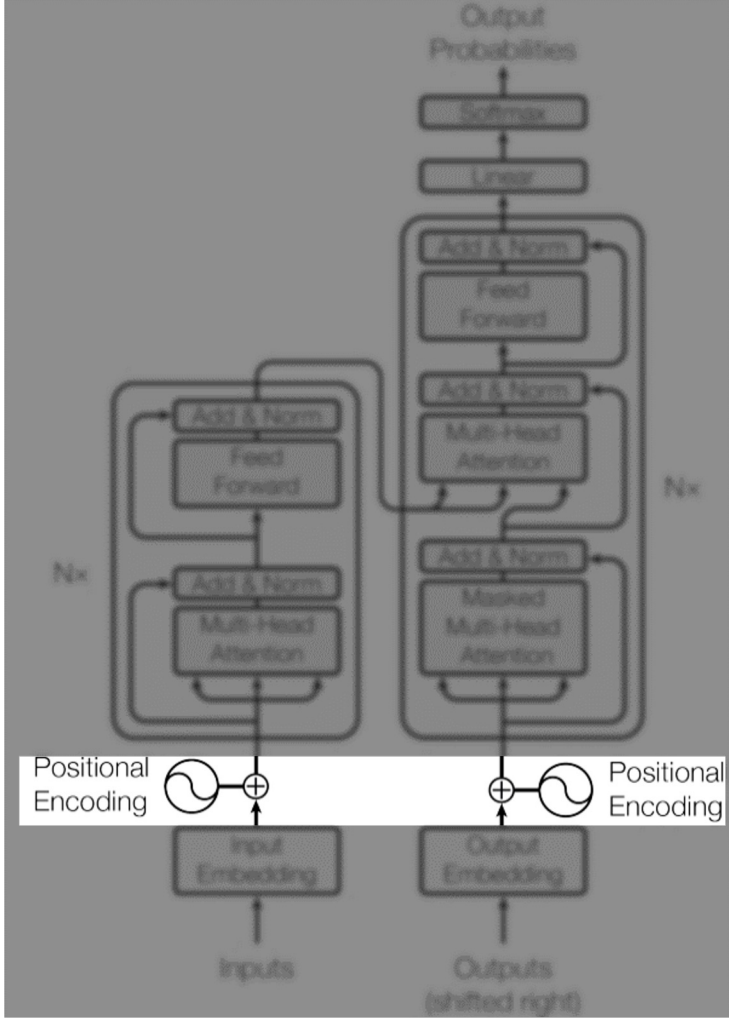


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

What are these Nx blocks?

- The Nx blocks **represent** the number of **stacked layers** in both the **encoder** and **decoder** of a transformer.
- Each layer contains several **key subcomponents**, and the **organization** of these **blocks** defines the **core functionality** of the model.
- If **Nx = 6**, there are **6 identical layers** where each token goes through multi-head attention, residual connections, and feed-forward networks, refining its representation

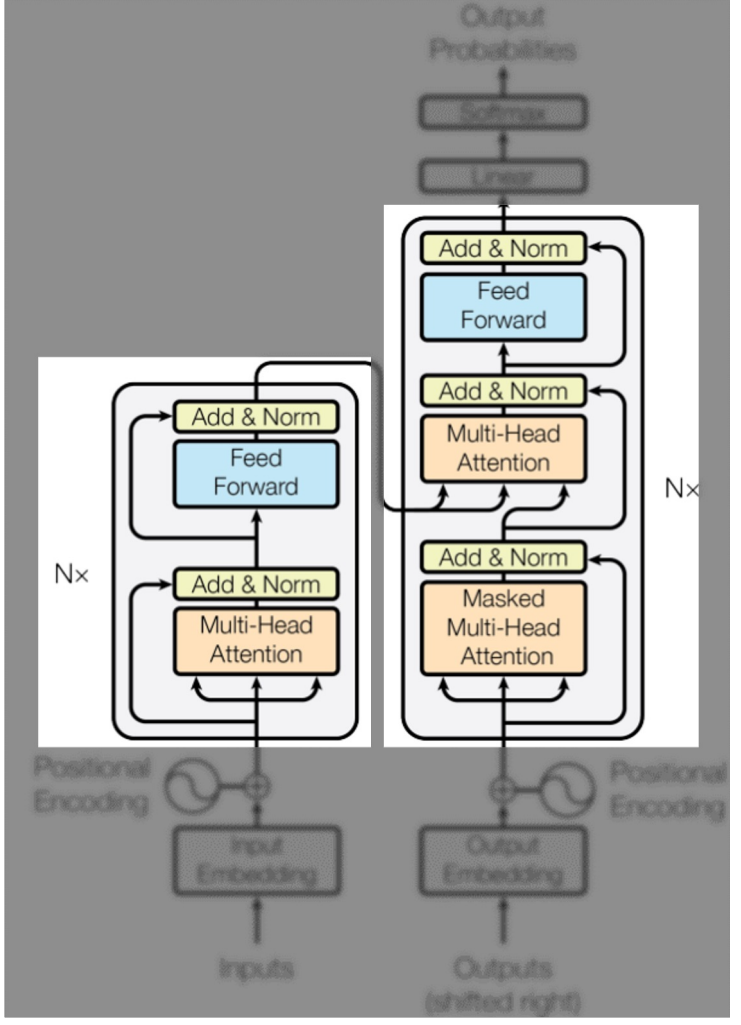


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

What makes an encoder/decoder?

- To explain why one block acts as an encoder and the other as a decoder we must turn our attention to attention.

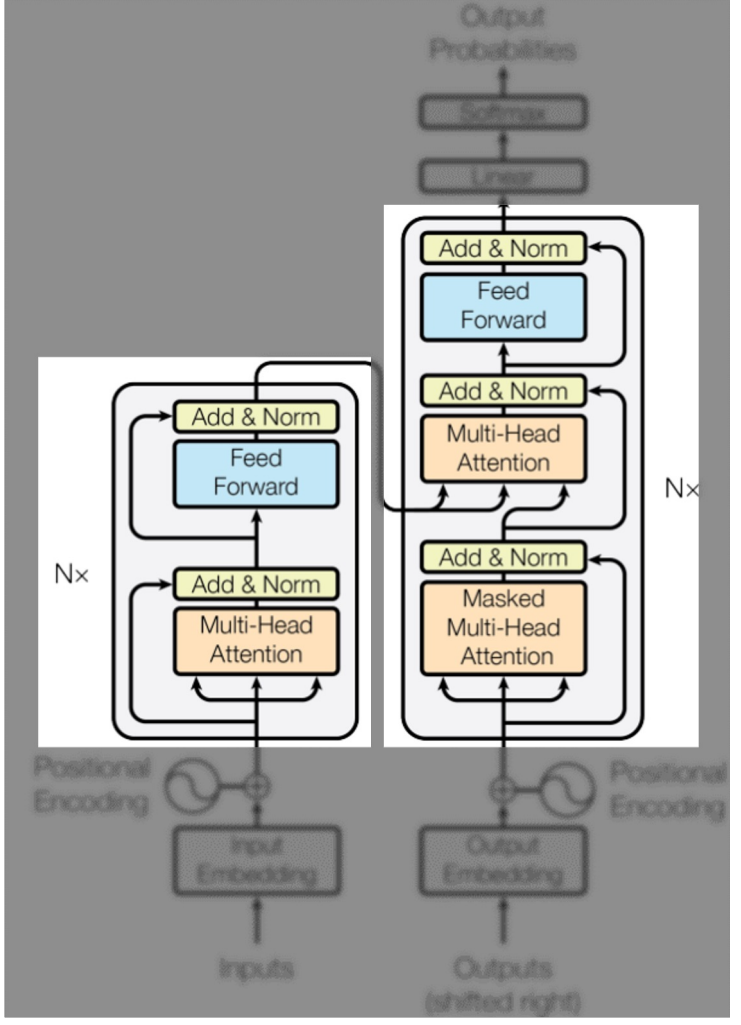


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention

- What happens when our sentences get to this block?
  - [‘i’, ‘am’, ’not’, ‘a’, ‘purple’ , ‘ballon’]
- Which are embeddings at this point, but we’ll use words.
- Now, they enter the **multi-head self-attention** mechanism.
- The **self-attention mechanism** allows each word (token) to attend to every other word in the sentence to learn which words are most relevant to understanding the current token.

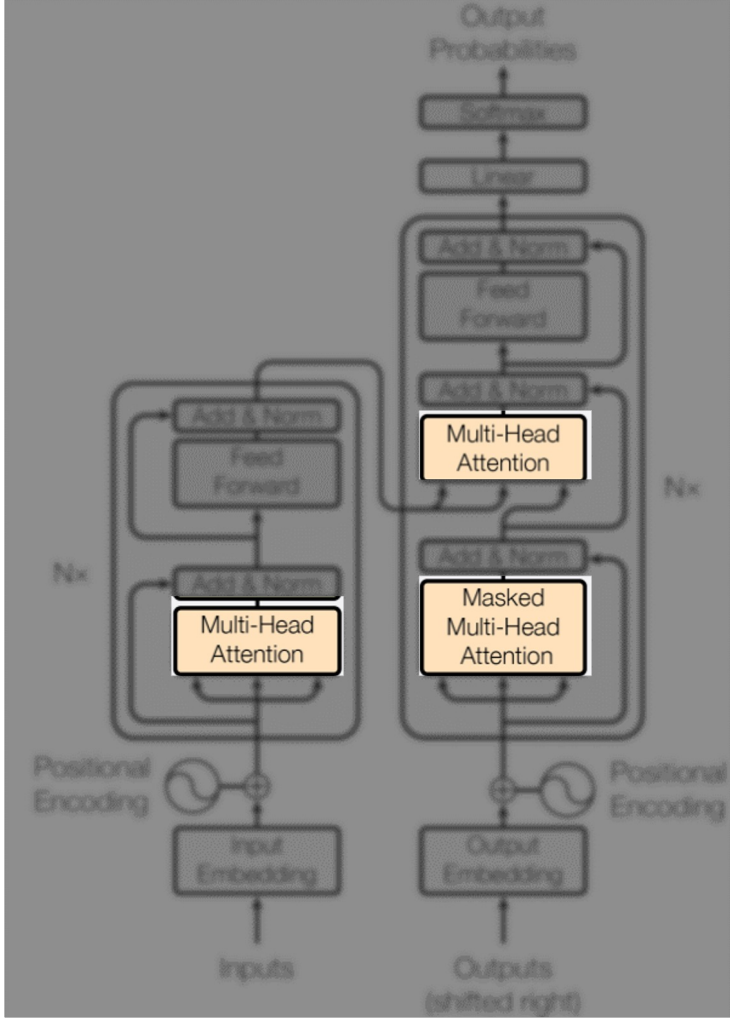


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention – Q,K,V

- When processing the token "**purple**", the model might decide that "**balloon**" is important because the phrase "**purple balloon**" makes sense together.
- The way this is handled is by transforming each embedding into three vectors.
  - **Query**(Q) -> Word we focus on (purple)
  - **Key**(K) -> Another word being compared
  - **Value**(V) -> information about each word that will be weighted to the attention score.

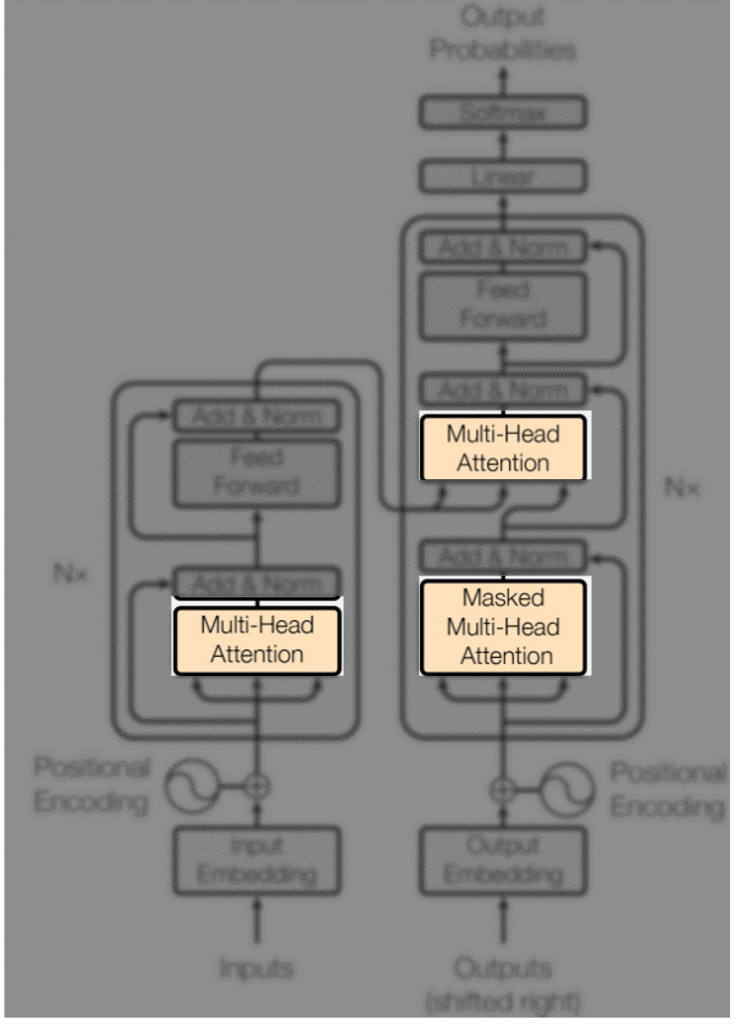


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention – Q,K,V details

- In the attention mechanism, the Query (Q), Key (K), and Value (V) vectors are derived from the **same input embeddings** through **learned linear transformations**.
- Here's how they are determined:
  - Input Embeddings: Each word in a sentence is initially represented by an embedding vector.
  - Weight Matrices: Three different weight matrices (W_q, W_k, W_v)
- These weight matrices are used to transform these embeddings into the Q, K, and V vectors.
- These matrices are learned during the training process.

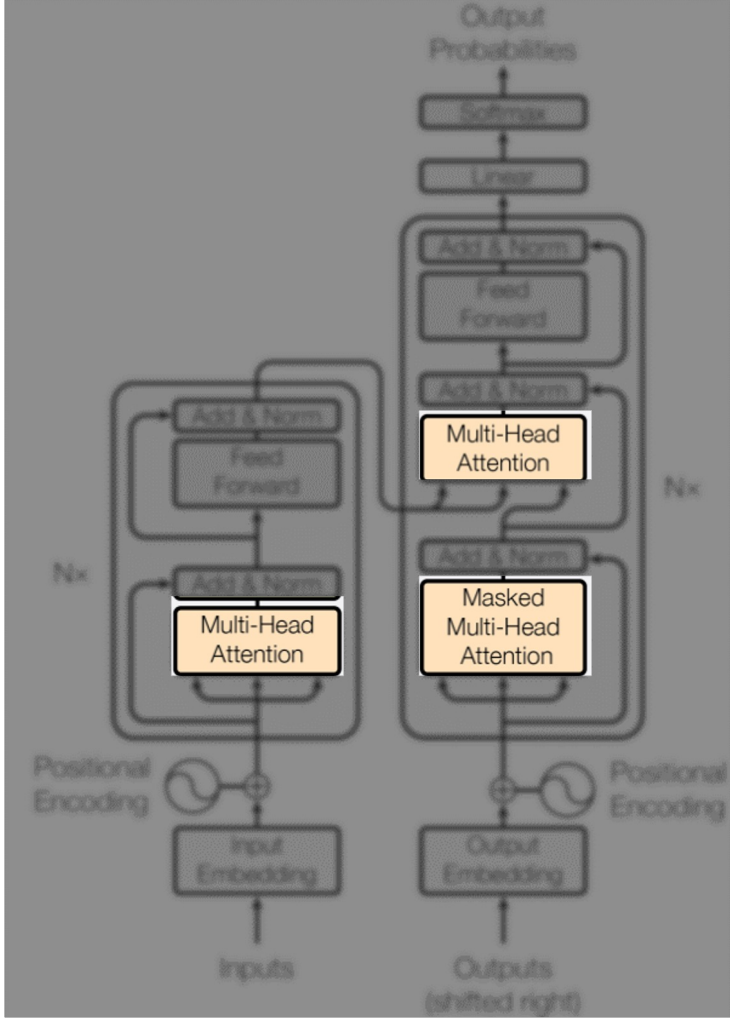


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention – Example Transformation to Q, K, and V

- [‘i’, ‘am’, ’not’, ‘a’, ‘**purple**’ , ‘ballon’]

Word	Final embedding
Purple	[0.7, 0.6, 0.8, …, -0.4]

1. Transformation into Q, K, V, vector
   - The embedding is multiplied by a weight matrix $\mathbf{W}^k$
   - The embedding is multiplied by a weight matrix $\mathbf{W}^q$
   - The embedding is multiplied by a weight matrix $\mathbf{W}^V$

Word	K vector	Q Vector	V vector
Purple	[0.25, -0.15, 0.33, …, 0.5]	[0.30, -0.35, 0.42, …, 0.2]	[0.65, 0.103, 0.333, …, 0.5]

Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention – The process

- **Calculate Attention Scores**:
  - For each word, the Query (Q) vector of the current word is compared to the Key (K) vectors of all other words in the sentence to compute attention scores.

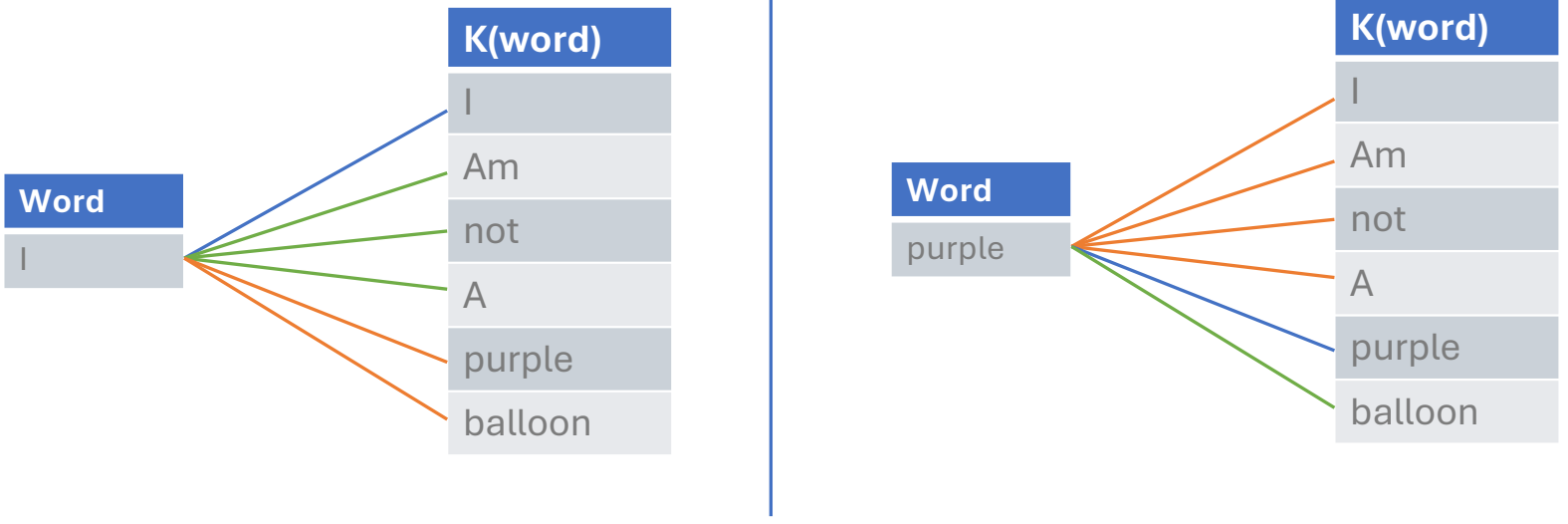


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Attention - Output

- **Key (K)** vectors are used for comparing words to determine attention scores.
- **Attention scores** are computed dynamically during the attention process.
- **Value (V)** vectors contain the actual content that gets weighted by the attention scores.
- The weighted sum of the **Value (V) vectors** is the final output of the attention process.

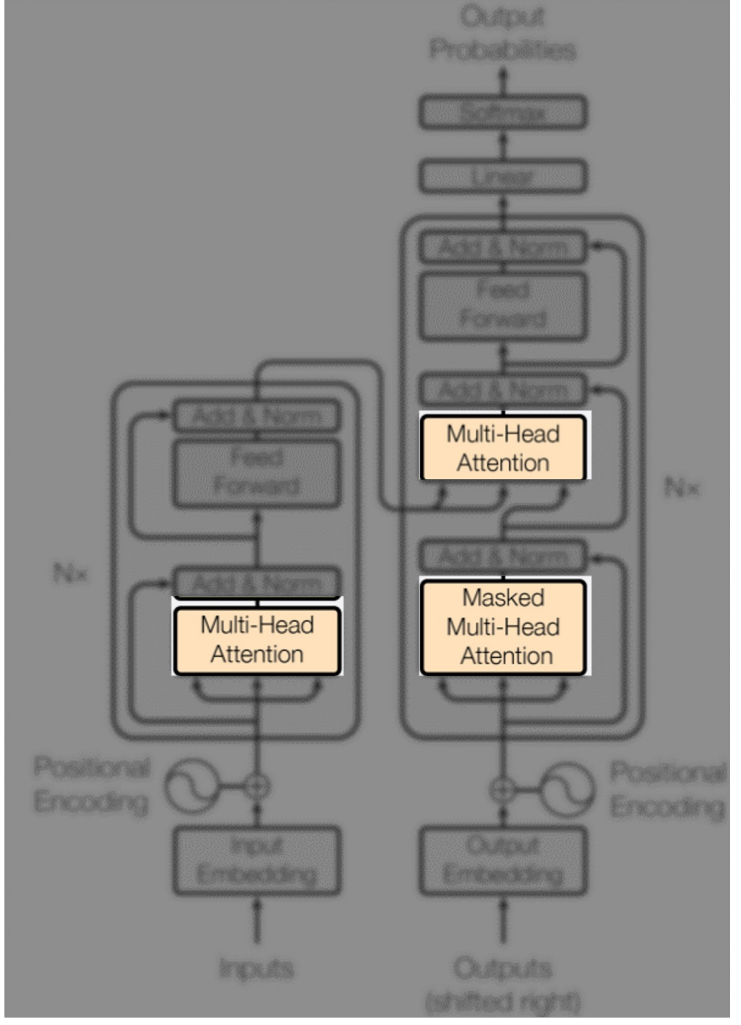


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Masked Attention

- **Purpose**: Prevents the model from “peeking” at future tokens during training, ensuring that the output is generated sequentially
- **Mechanism**:
  - A mask is applied to the attention scores to hide future words.
  - For example, when processing "purple," the model cannot attend to "balloon" or any subsequent words.

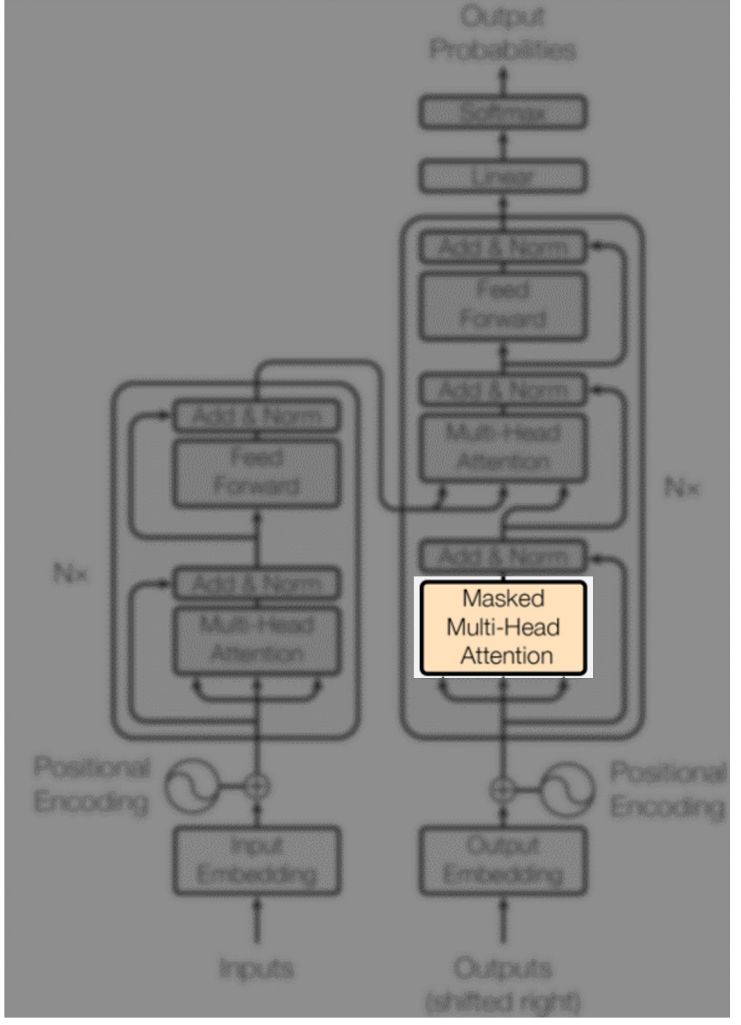
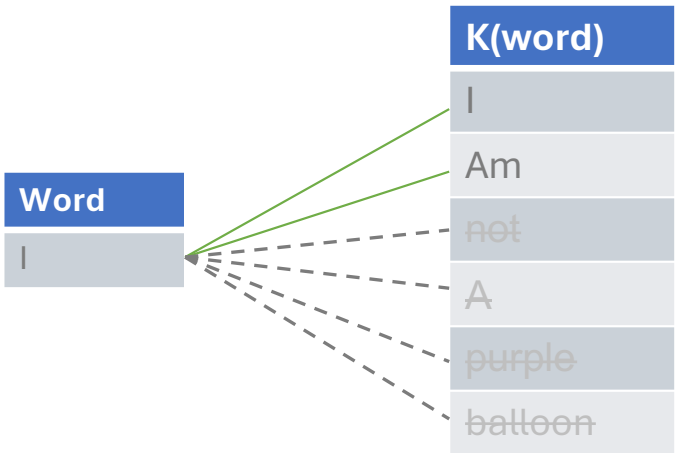


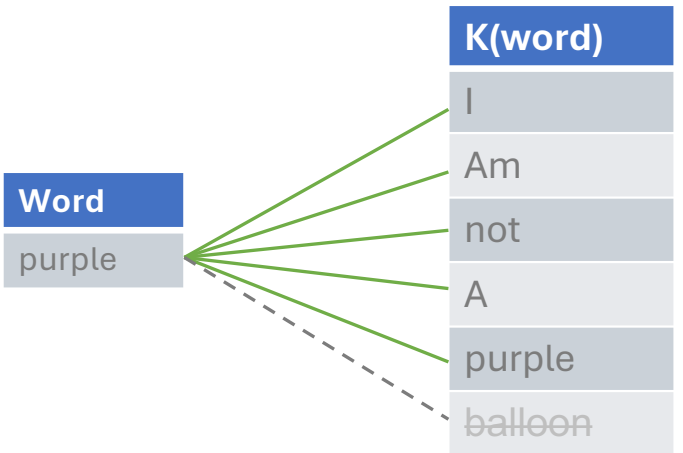
Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Masked Attention

Current Word: “I”
Masked Words:
- [“not”, ”am”, ”a” ,”purple”, “balloon”]



Current Word: “I”
Masked Words:
- [“balloon”]



Masked Attention: How and why

- **Purpose**:
  - Prevents the model from looking at future tokens when generating text or performing autoregressive tasks.
- **Mechanism**:
  - A mask is applied to the attention scores to hide future words.
  - For example, when processing "purple," the model cannot attend to "balloon" or any subsequent words.
- **Effect**:
  - During text generation, the model can only rely on previously generated words, making predictions more realistic and aligned with the goal of sequential generation.

Ultimately if your goal is text generation, you don't want to give the model the words you are trying to generate.

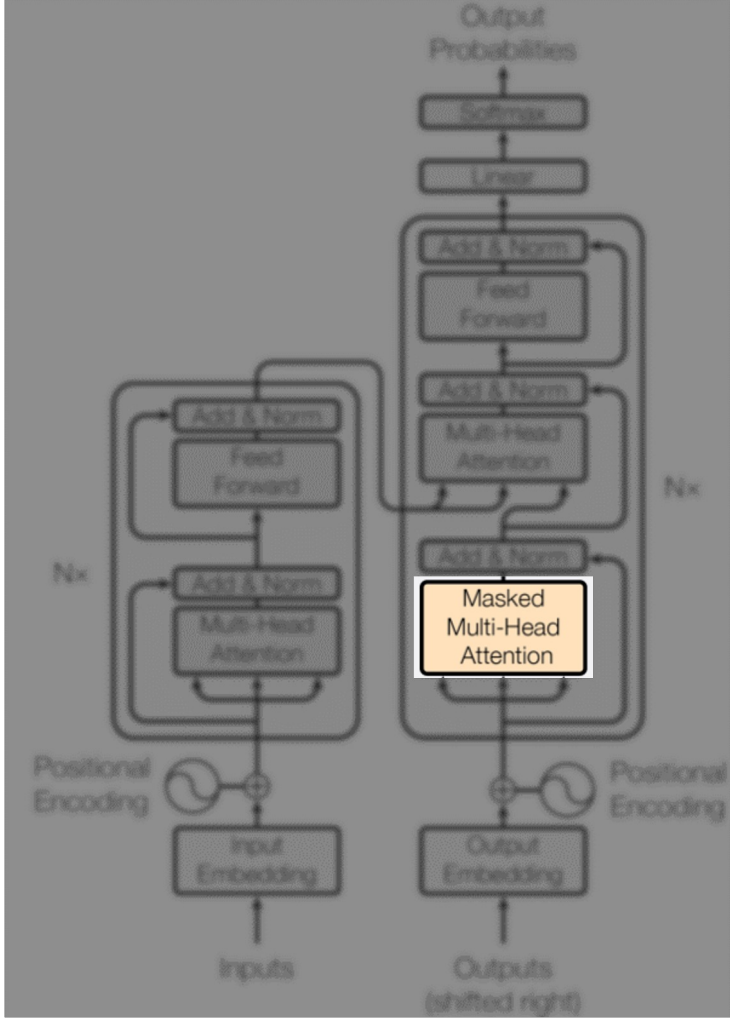


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Masked Attention: Decoder

- Use Case: Essential in **autoregressive** models where a **unidirectional** flow of information is required, such as in generating text sequences.
- A decoder is a component of a Transformer model that contributes to it's autoregressive model.

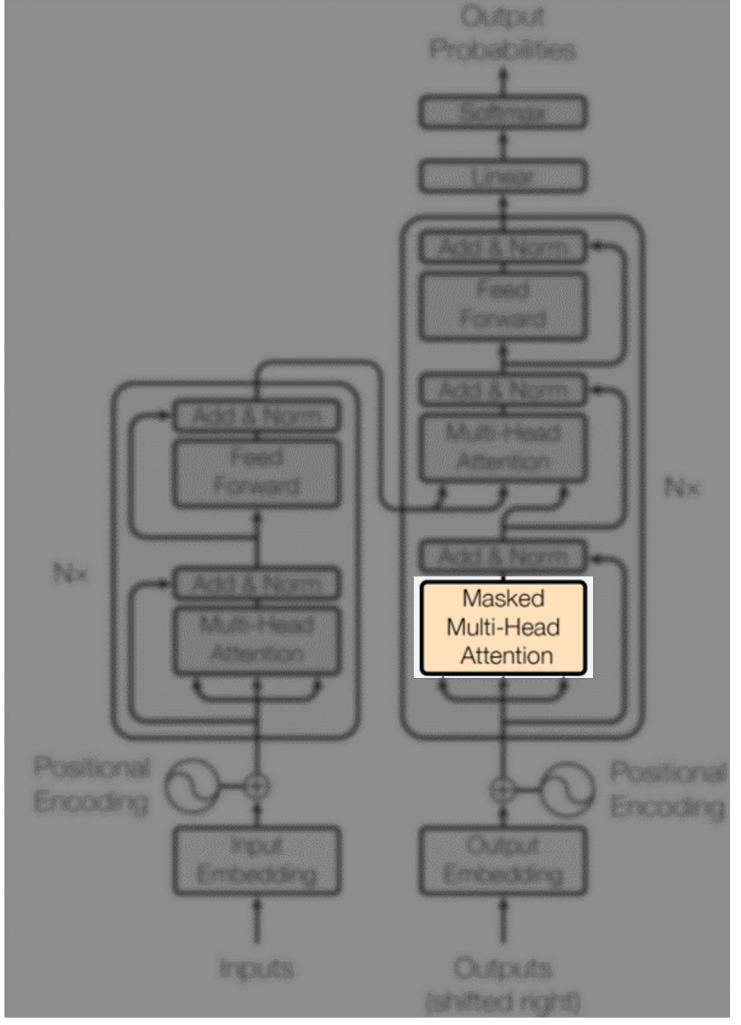


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformers – Add and norm

- **Add & Norm**
  - **Residual Connection:** This involves adding the input of a layer to its output. It helps in preserving information from earlier layers and aids in gradient flow during training, preventing vanishing or exploding gradients.
  - **Layer Normalization:** After adding the residual connection, layer normalization is applied. This normalizes the outputs across different channels to stabilize and speed up training by maintaining a consistent scale for the outputs

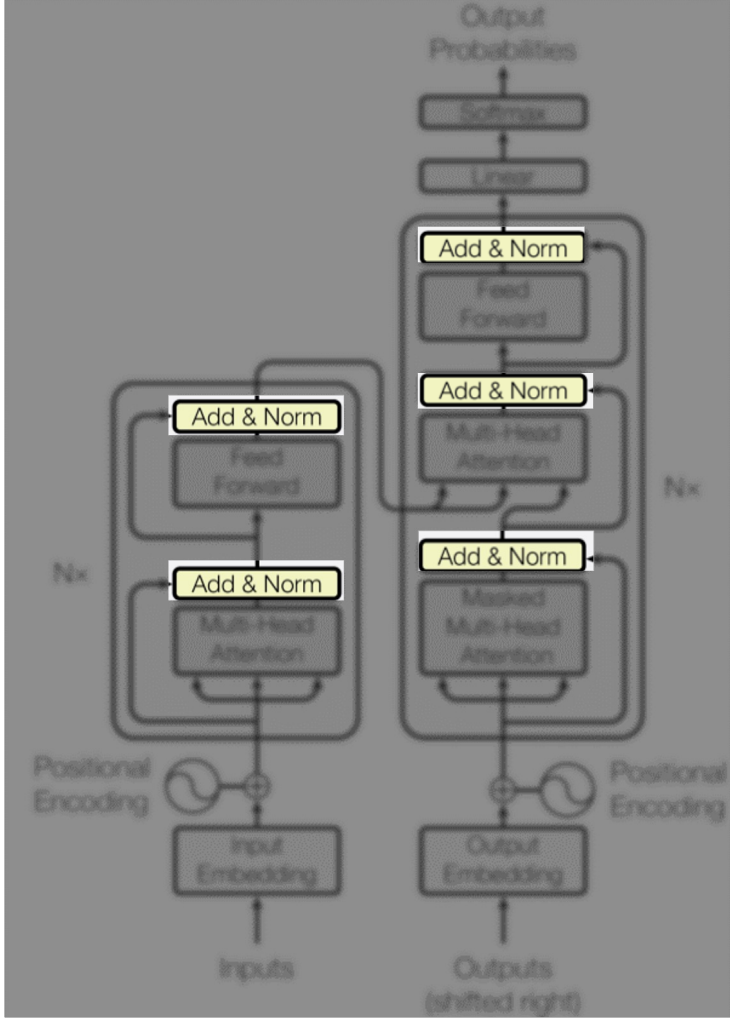


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformers – FeedForward

FeedForward Layer

- **Structure**: Consists of two dense (fully connected) layers with a ReLU activation in between. Each position in the sequence is processed independently and identically.

- **Purpose**: Transforms the representation of the input sequence into a more suitable form for the task at hand. It helps in capturing complex patterns by applying non-linear transformations

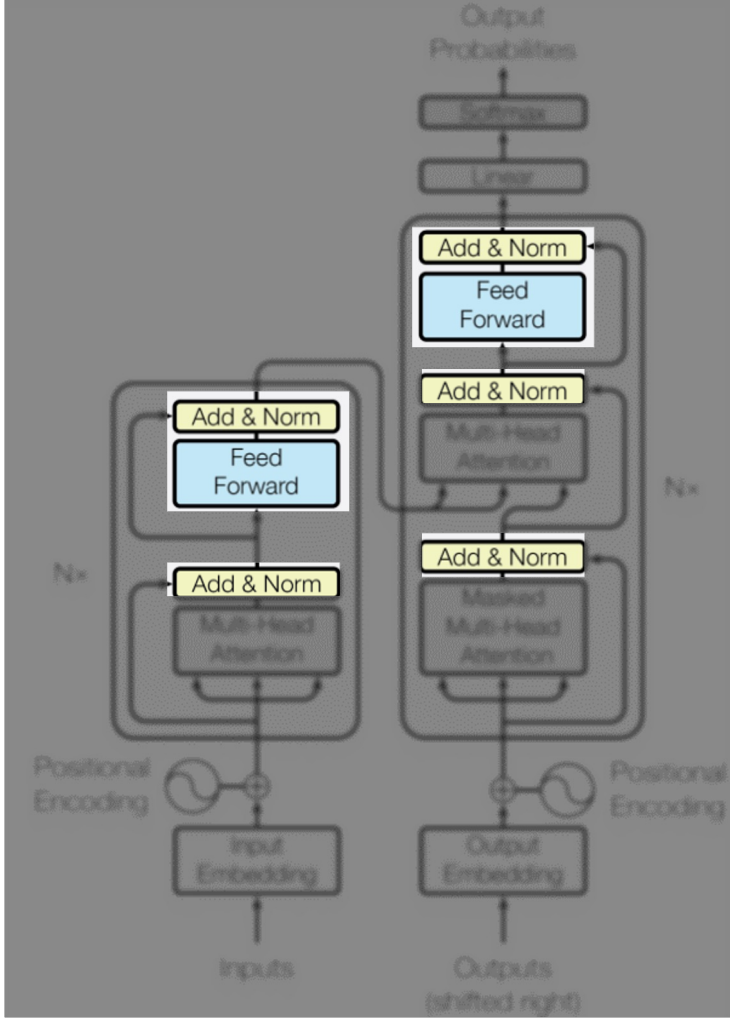
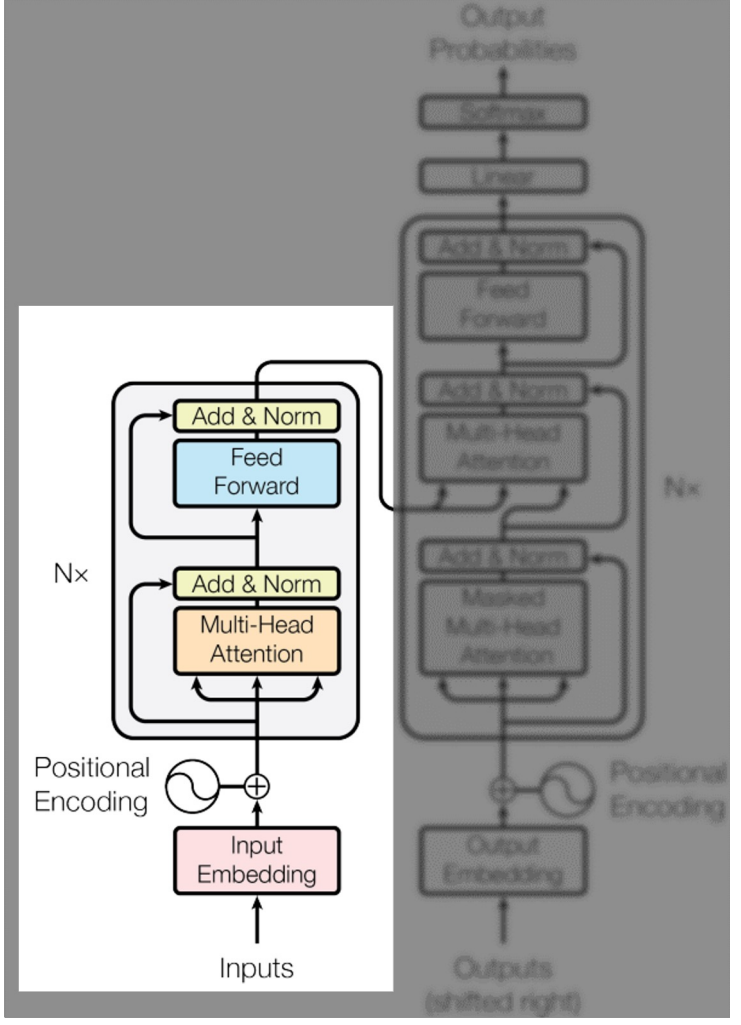


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Transformers: Encoder Summary

- Inputs:
  - **Input Embeddings:** Converts each word in the input sequence into a dense vector representation.
  - **Positional Encoding:** Added to word embeddings to provide information about word order.
- Outputs:
  - **Encoded Representations:** A set of vector representations that capture the contextual meaning of the input sequence, ready for the decoder to use.



Transformers: Decoder Summary

- Inputs:
  - **Input Embeddings**: Converts each token in the target sequence into embeddings, shifted right.
  - **Positional Encoding**: Added to output embeddings to maintain order information.
- Encoded Representations from Encoder:
  - Used in cross-attention layers to incorporate context from the input sequence.
- Outputs:
  - **Generated Sequence**: Produces the final output sequence, one token at a time, using masked attention to ensure predictions are based only on past and present tokens.

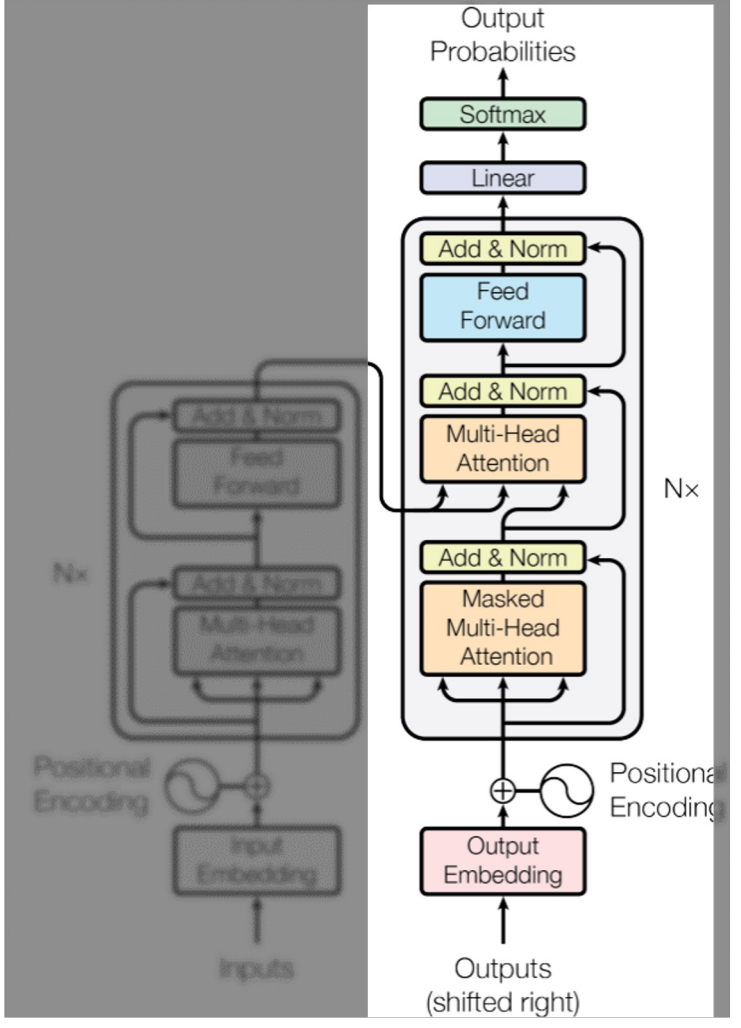


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Positional Encoding Process

- Process
  - Generate Unique encoding for each position
  - Add encoding to token embeddings
- Formula
  - $PE(pos, 2i) = \sin(\frac{pos}{10000^{2i/d}})$
  - $PE(pos, 2i+1) = \cos(\frac{pos}{10000^{2i/d}})$
- Implementation
  - Create matrix of position-dimension pairs
  - Apply sine to even indices, cosine to odd indices
  - Add resulting encodings to token embeddings

- Benefits
  - Preserves sequence order information
  - Enables attention mechanism to consider token positions
  - Generalizes to sequences of varying lengths

Attention: Quick recap

- Core innovation of Transformers is **attention**
- **Attention** allows the model to focus on different parts of a sequence simultaneously.
- **Key components**
    - **Query (Q)**: What we're looking for
    - **Key (K):** What we must match against
    - **Value (V):** The actual content to retrieve
- **Process**
    - **Calculate similarity** scores between the **Query (Q)** and each **Key (K)** by taking the dot product.
    - **Apply SoftMax** to the similarity scores to convert them into attention weights.
    - **Use the attention weights** to compute a weighted sum of the **Values (V),** focusing on the most relevant parts of the input.

$$\mathbf{A}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Attention: Step by step

- Step 1: **Input Definitions**
  - **Query (Q)**: Represents what the model is trying to find in the sequence (e.g., a word or a position in a sentence).
  - **Key (K)**: Represents each position in the sequence that the query will be compared to.
  - **Value (V)**: Contains the actual content or information at each position in the sequence.

- Step 2**: Calculate Dot Products (Similarity Scores)**
  - For each query Q, calculate its similarity with each key K by computing their dot product:
  - $Score(Q, K) = Q * K^T$

- Step 3: **Scale the scores**
  - To prevent the dot product values from becoming too large (especially in high dimensions), the scores are scaled by the square root of the dimension of the key, d_k
  - $Scaled\ Score = \frac{Q \times K^T}{\sqrt{d_k}}$

Attention: Step by step

- Step 4: **Apply SoftMax**
  - Apply the SoftMax function to the scaled scores to convert them into probability-like attention weights:
  - $Attention\ Weights = softmax\left(\frac{Q \times K^T}{\sqrt{d_k}}\right)$
- Step 5: **Compute Weighted Sum of Values**
  - $Output\ = \sum(Attention\ Weights\ \times V)$
- Step 6: **Final Output**
  - The result is a vector (or set of vectors) that represents the input, with more focus on the positions deemed important by the attention mechanism.

LLMs fall into 2 Categories.

Autoregressive Models:

- Predict the next token in a sequence based on the previous tokens (e.g., **GPT-3**).
- These models generate text one token at a time, using prior outputs as input for the next step.
- Autoregressive models are typically **decoder-only**.

Conditional Generative Models:

- Generate text based on given input or context (e.g., **T5**).
- Often used for tasks like **translation**, **summarization**, and **question answering**, where the output is conditioned on the input data.
- They can be **encoder-decoder** (e.g., T5) or **encoder-only** (e.g., BERT, used for understanding tasks).

BERT

BERT

- **Encoder-Only Model**: Focuses on understanding the entire input sequence by looking at tokens in both directions (bidirectional).
- **Tasks**: Ideal for tasks like text classification, question answering, token classification and named entity recognition.

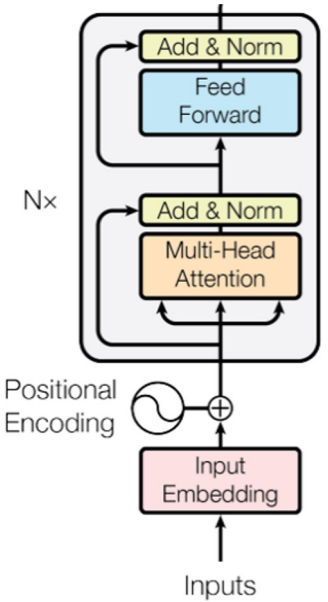


Fig: The Transformer - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Why is BERT less common for LLMs?

- **Encoder-Only Architecture**
  - BERT is an encoder-only model, designed for understanding tasks (e.g., classification, question answering), but not inherently suitable for generative tasks like text generation or completion, which LLMs often focus on.
- **Bidirectional Masking**
  - BERT uses bidirectional masking, predicting missing tokens based on both left and right context. While great for understanding, it makes BERT less suited for tasks requiring sequential token generation (e.g., summarization, dialogue generation).
- **Limited Generation Capabilities**
  - BERT is primarily extractive, making it less useful for generative tasks like content creation, where models like GPT excel.
- **Context Window**
  - BERT's context window is smaller compared to models like GPT-3, limiting its ability to handle very long text sequences in LLM applications.
- **Specialization**
  - BERT is often fine-tuned for specific tasks, whereas LLMs like GPT are more generalized and versatile for a broader range of applications (e.g., text generation, conversation).

GPT

GPT

- **Decoder-Only Model**: Generates text by predicting the next token in a sequence, using context from previously generated tokens (unidirectional).
- **Tasks**: Ideal for tasks like text generation, language modeling, and text completion.

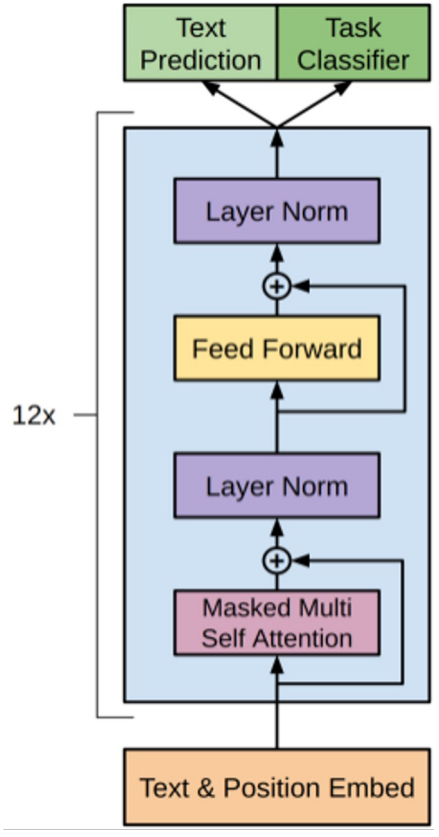


Fig: GPT - Model Architecture, Vaswani et al., Attention Is All You Need (2017).

Production Large Language Models - Examples

- **Two key attributes:**
  - **Context Window**: The amount of text the model can process at once.
  - **Parameter Size**: The number of learnable weights, which impacts the model's capacity and performance.
- **Meta's LLaMA Models:**
  - Lama 3.1 Comes in 8B, 70B, or 405B parameter sizes
  - With a context window of up to 128k tokens.
- **OpenAI's GPT-3.5:**
  - 175B parameters.
  - Context window of 4k Tokens or 16k for turbo variant
- **OpenAI's GPT-4.0:**
  - Rumored to have 1 trillion+ parameters
  - Context window of 128K tokens

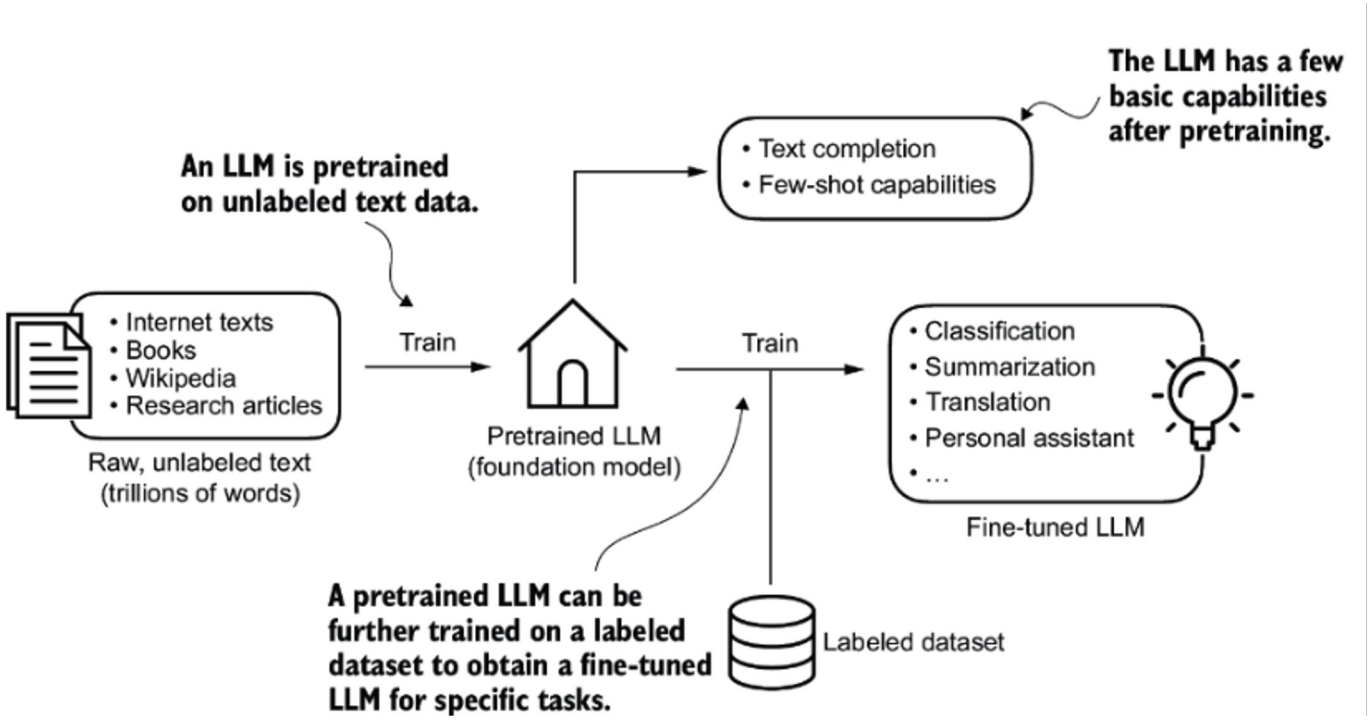
Parameter: A learnable weight in a model that adjusts during training to capture patterns in the data, influencing the model's performance and capacity.

Token: A unit of text (word, subword, or character) that the model processes. Tokens are the building blocks of the input text for language models.

How LLMs are created

- **Transformer Architecture**: LLMs are commonly built on a transformer architecture.
- **Training Scale**: **A model** becomes an **LLM** after being trained on **massive datasets** and having a large many parameters.
- **Foundation of LLMs**: Understanding how to build a transformer provides a strong foundation for creating an LLM.
- **Fine-Tuning**: In practice, most LLMs are created by fine-tuning existing models rather than training from scratch.
- **Challenges of Training from Scratch**: While possible, creating an LLM from scratch requires an enormous amount of data and computational resources, which is often prohibitive.

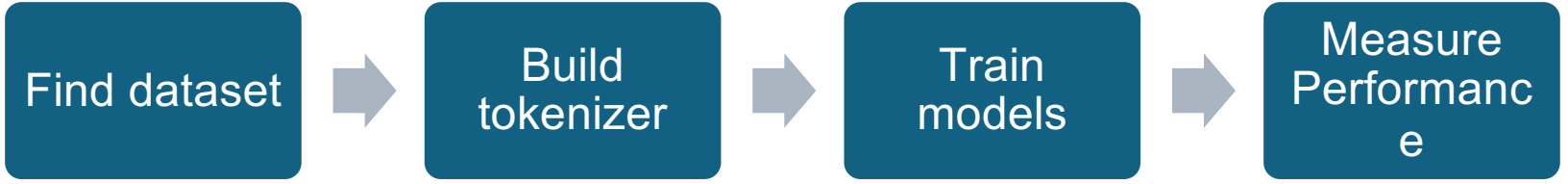
High level overview of LLM training



Pretraining an LLM, *Sebastian Raschka*, *Build a Large Language Model (From Scratch)*.

Creating an LLM from scratch – High level overview

- Creating a LLM entirely from scratch can be challenging.
  - You need to find a massive and high-quality dataset
  - You need to build a tokenizer
  - You need to either train a model from scratch or fine-tune a pre-trained model like GPT2
  - Then you train, evaluate and fine tune.



Finding a dataset - Considerations

- **Find data:** Determine what kind of data you need for your use case or domain.
- Look for existing corpus's or other open-source datasets
  - C4 – Web crawl data
  - BookCorpus – Collection of indie books
  - P3 - Public Pool of Prompts
- **Find your own data**
  - Wikipedia is always an option
  - Online posts(blogs, forums, social media)
- Remember diverse data is key for general-purpose LLMs.

- **Processing:** Determine how to combine, clean and the process data
- Common tasks include:
  - Sanitization
  - Quality filtering
  - De-duplication
  - Privacy redaction
  - Balancing

Tokenization

- Tokenization is the process of breaking text into smaller units (tokens)
- Essential first step in processing text for LLM training
- There are different tokenization types you can use.
- There are different algorithms you can use as well.

Steps to build a tokenizer

- Analyze your dataset
- Choose a tokenization algorithm (Tokenization types)
- Train the tokenizer on your corpus
- Test and refine the tokenizer

Tokenization Types

Word-level

- Splits text into individual words.
- Example: "Tokenization is fun" → ["Tokenization", "is", "fun"].
- Limitation: Can lead to large vocabularies and struggles with unseen words (out-of-vocabulary words)

Subword-level (most **common** for LLMs)

- Breaks words into smaller subword units.
- Example: "Tokenization" → ["Token", "ization"].
- Useful for handling rare words and reducing the size of the vocabulary.
- Techniques: Byte-Pair Encoding (BPE), WordPiece

Character-level

- Splits text into individual characters.
- Example: "Tokenization" → ["T", "o", "k", "e", "n", "i", "z", "a", "t", "i", "o", "n"].
- Simplifies vocabulary, but sequences can become very long.

Tokenization Algorithms

Byte Pair Encoding (BPE)

- Merges the most frequent pairs of bytes or characters into subwords.
- Iteratively reduces a large vocabulary into manageable subword units.
- Used in models like GPT-2 and RoBERTa.

WordPiece

- Similar to BPE, but optimizes based on likelihood, not frequency.
- Splits words into subwords, prioritizing parts that appear frequently in training data.
- Used in BERT and ALBERT.

SentencePiece

- A tokenizer that treats input text as a sequence of raw bytes without requiring space as a delimiter.
- Can tokenize without pre-segmentation, making it effective for languages without spaces between words (e.g., Chinese).
- Used in models like T5 and XLM-R.

Writing your own GPT

- While it would be challenging, a simple GPT architecture is just an encoder and decoder.

1. **Design the Architecture**:
   - Implement the encoder-decoder structure using frameworks like PyTorch or TensorFlow.
   - Focus on self-attention mechanisms and transformer layers.
2. **Train the model**
   - Use the dataset to train the model from scratch, focusing on predicting the next word in a sequence.
   - Ensure robust training with appropriate hyperparameters and regularization techniques.

Note:

- Realistically most advice you would receive would be to start from GPT2 as OpenAI has made those weights available.
- Using GPT2 would also allow you to use the GPT2 tokenizer.
- Hugging face allows for quickly building and using models

Huggingface (Transformers) code examples.

- Tokenizer code example.

```
from transformers import AutoTokenizer
python_code = r"""My name is robert"""
tokenizer = AutoTokenizer.from_pretrained("gpt2")
print(tokenizer(python_code).tokens())

#output
['My', 'Ġname', 'Ġis', 'Ġro', 'bert', 'Ċ']
```

- Loading gpt2 weights

```
from transformers import AutoConfig, AutoModelForCausalLM, AutoTokenizer
config = AutoConfig.from_pretrained("gpt2-xl", vocab_size=len(tokenizer))
model = AutoModelForCausalLM.from_config(config)
model.resize_token_embeddings(len(tokenizer))
```

- Regardless if you create your own model or use gpt2 the next step would be fine tuning.

The process of fine tuning an LLM

Goal: Adapt pre-trained model to specific tasks or domains

Find Base Model	Gather Data for Fine-Tuning	Freeze Layers	Fine-Tune the Model	Evaluate and Test
• Choose a pre-trained LLM (e.g., BERT, GPT variant) that fits your task.	• Collect domain-specific data to adapt the model to your specific use case.	• Freeze most of the model's layers to retain general knowledge, while leaving specific layers (e.g., last few layers) unfrozen for fine-tuning.	• Train the model on the new data, adjusting only the unfrozen layers to adapt the model without overfitting.	• Validate the model's performance on task-specific data, adjust hyperparameters if necessary, and test on unseen data.

Base model Selection

Use Case Considerations:

- Task specificity (e.g., text classification, named entity recognition, sentiment analysis)
- Domain relevance (e.g., medical, legal, general purpose)
- Language requirements (monolingual vs. multilingual)

Model Architecture options:

- BERT(Bidirectional Encoder Representations from Transformers)
- RoBERTa (Robustly Optimized BERT Approach)
- GPT (Generative Pre-trained Transformer) family
- T5 (Text-to-Text Transfer Transformer)

Base model Selection Example

```
# Load the pre-trained BERT tokenizer and model
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = TFBertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)  # Binary classification

# Check model summary
model.summary()
```

Fine-tuning data

Purpose:

- After selecting a base model, choose appropriate data for fine-tuning

Data Selection Process:

- Collect task-specific data with relevant labels or annotations
- Ensure data quality, diversity, and proper formatting

Tokenize and load data

- Once you have new data, preprocess it appropriately and use it to train your model on a new task.

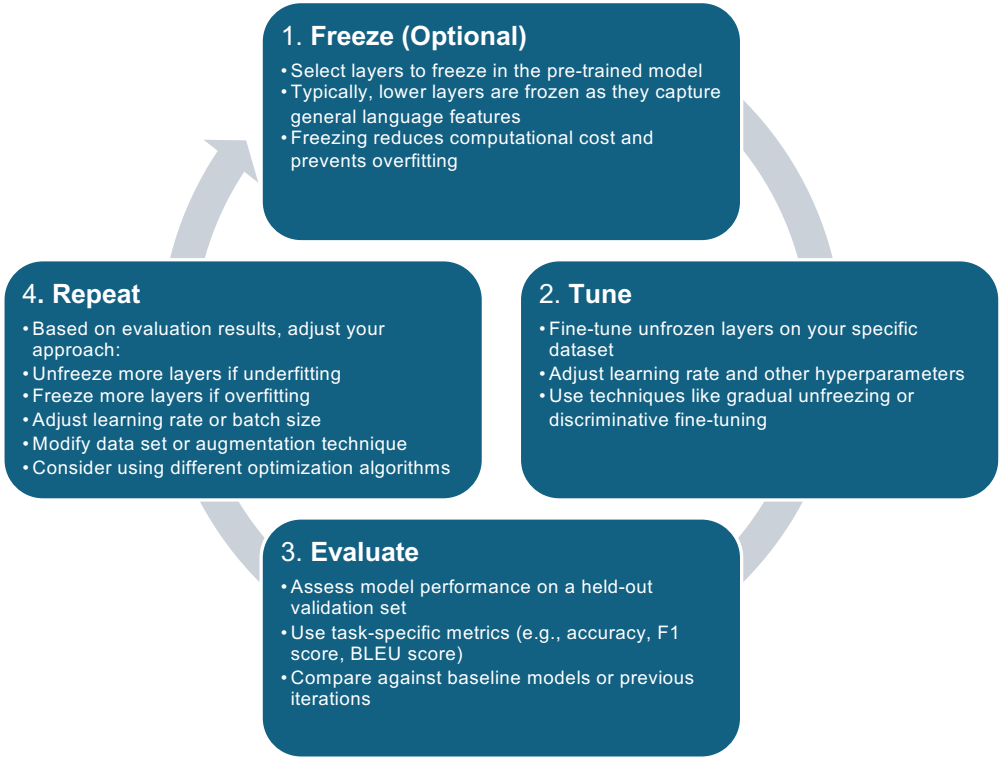
Sentiment Analysis Example

- Collect text samples with sentiment labels (e.g., positive, negative, neutral)
- **Example**: "This product exceeded my expectations!" (Label: Positive)

Common Fine-tuning Tasks:

- Sentiment analysis
- Named entity recognition
- Question Answering
- Text Classification
- Machine translation
- Text Summarization

Freeze, tune, evaluate and repeat



Freeze layers code example

```
# Freeze all layers except the last few by setting `trainable=False`
for layer in model.layers[:-2]:  # Freeze all layers except the last two
    layer.trainable = False

# Print model summary to verify which layers are frozen
model.summary()
```

- Most frameworks use **trainable** as the parameter to mark a layer frozen or not
- If **trainable** is **True**, that means the layer is **not frozen** and weights can be adjusted during training.

Intuition behind freezing layers

- **Knowledge storage in Neural Networks:**
  - The knowledge of any deep learning network is primarily stored in its weights
  - These weights are distributed across the network's layers

- **Layer Hierarchy in Learning:**
  - Think of deep learning as a process of knowledge distillation.
  - Initial layers learn more general, low-level features
  - Later layers capture more specific, high-level features and task-related knowledge

- **Rationale for Freezing:**
  - Freezing initial layers preserves general language understanding and learned low-level features.
  - Helps maintain the model's general language capabilities while specializing for the task.

Quick Note: Transfer Learning vs Fine-Tuning

- **Transfer Learning**
  - Takes a pretrained model and applies it to a new, typically smaller dataset or task
- **Approach**
  - You use the pre-trained model as a feature extractor, freezing most or all of the model's layers and only training the final layers specific to the new task.

- **Fine-tuning**
  - Specific type of transfer learning
  - Pre-trained model is further trained (tuned) on the new dataset,
  - Instead of freezing most of the layers, you allow some (or all) of them to be updated.
- **Approach**
  - Unfreeze some layers (often the deeper ones), then train the model on the new task with a smaller learning rate
  - This allows the model to adapt better to the new dataset while retaining some its previous knowledge.

Fine-Tuning

```
for layer in model.layers[:-5]:  # Unfreeze all but the last 5 layers
    layer.trainable = True

# Recompile the model with a lower learning rate for fine-tuning
model.compile(optimizer=Adam(learning_rate=1e-5), loss=SparseCategoricalCrossentropy,
metrics=['accuracy'])

# Continue fine-tuning the model
history = model.fit(dataset, epochs=3, validation_data=val_dataset)
```

- **Fine-Tuning:** can consist of changing the learning rate, freezing less or more layers, adjusting the dataset through augmentation or k-fold cross-validation.
- **Batch size adjustment**: Smaller batch sizes are often used in fine-tuning to allow for more frequent weight updates and can improve generalization.
- **Regularization:** Techniques like weight decay or dropout may be adjusted or added to prevent overfitting during fine-tuning.

Metrics and performance

- **Perplexity**: Measures how well the model predicts a sample. Lower perplexity indicates better performance.
- **Accuracy**: Used for classification tasks to measure the percentage of correct predictions.
- **F1 Score**: Combines precision and recall, useful for tasks with imbalanced data (e.g., token classification, informatio).
- **BLEU Score**: Evaluates the quality of machine-generated translations compared to human translations (used for translation tasks).
- **ROUGE Score**: Measures overlap between generated and reference text (used for summarization tasks).
- **Latency**: Measures how fast the model generates or processes text, crucial for real-time applications (Chatbots).
- **Memory and Computation**: Evaluates the model's resource efficiency, considering parameter size, context window, and computational costs.

Metric - Perplexity (PPL)

- Perplexity (PPL) is one of the most common metrics for evaluating language models.
- Perplexity is defined as the exponentiated average negative log-likelihood of a sequence
- PPL(X) = $\exp\left\{-\frac{1}{t}\sum_i^t \log p\theta\left(x_i|x_{<i}\right)\right\}$

```
# Example predictions (logits from the model) and true labels
y_true = np.array([1, 0, 0, 1])  # Ground truth labels
y_pred_logits = np.array([[0.1, 0.9], [0.7, 0.3], [0.6, 0.4], [0.8, 0.2]])  # Logits from model
# Compute categorical cross-entropy loss
cross_entropy_loss = tf.keras.losses.sparse_categorical_crossentropy(y_true, y_pred_logits,
from_logits=True)
# Compute perplexity (e^loss)
perplexity = tf.exp(tf.reduce_mean(cross_entropy_loss))
# Print perplexity
print(f"Perplexity: {perplexity:.4f}")
```

Metric – BLEU Score

- The **BLEU (Bilingual Evaluation Understudy)** score is commonly used to evaluate the quality of machine-generated translations. It compares the n-grams of the generated text to those of reference translations.

```
from nltk.translate.bleu_score import sentence_bleu
reference = [['my', 'name', 'is', 'robert', 'i', 'was', 'born',
'in', 'may']]
candidate = ['my', 'name', 'is', 'robert', 'i', 'was', 'born',
'in', 'orange']
score = sentence_bleu(reference, candidate)
print(score)
0.8633400213704505
```

Metric – BLEU Score algorithm

- **Input**:
  - Candidate string (or list of candidate strings for a corpus).
  - List of reference strings (or reference translations for a corpus).
- **n-gram precision**:
  - For each candidate, we compute the number of matching n-grams (unigrams, bigrams, etc.) with the reference translations.
  - Count the n-grams from the candidate that are also found in the reference translations.
- **Brevity Penalty**:
  - BLEU includes a brevity penalty to prevent short candidate translations from receiving high scores simply due to conciseness.
- **Overall Score**:
  - The BLEU score combines the modified n-gram precisions using a weighted geometric mean and applies the brevity penalty.

Metric – Rouge Score

- The **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** score is widely used to evaluate text generation tasks such as summarization. It measures n-gram overlap between the generated and reference summaries.

```python
from rouge_score import rouge_scorer

# Example reference and generated summaries
reference_summary = "My name is robert and i was not born in an orange."
generated_summary = "My name is robert and i was born in a purple orange."

# Initialize the ROUGE scorer
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)

# Calculate ROUGE scores
scores = scorer.score(reference_summary, generated_summary)

# Print the ROUGE-1, ROUGE-2, and ROUGE-L scores
print(f"ROUGE-1: {scores['rouge1'].fmeasure:.4f}")
print(f"ROUGE-2: {scores['rouge2'].fmeasure:.4f}")
print(f"ROUGE-L: {scores['rougeL'].fmeasure:.4f}")

Output:
ROUGE-1: 0.8333 ROUGE-2: 0.6364 ROUGE-L: 0.8333
```

- **ROUGE-1:** Measures overlap of unigrams (individual words) between the generated and reference text.
- **ROUGE-2:** Measures overlap of bigrams (pairs of consecutive words) between the generated and reference text.
- **ROUGE-L:** Measures the longest common subsequence (LCS) between the generated and reference text, capturing sentence-level structure and fluency.

Metric – Rouge Score Math

- **Rouge 1**
  - $RECALL = \frac{number\ of\ overlapping\ n-grams}{number\ of\ n-grams\ in\ reference}$
  - $PRECISION = \frac{number\ of\ overlapping\ n-grams}{number\ of\ n-grams\ in\ the\ candidate}$
  - $F1 = 2x\frac{Precision\ x\ Recall}{(Precision+Recall)}$
- **Rouge 2**
  - $RECALL = \frac{Number\ of\ overlapping\ bigrams}{Total\ bigrams\ in\ reference\ text}$
  - $PRECISION = \frac{Number\ of\ overlapping\ bigrams}{Total\ bigrams\ in\ generated\ text}$
  - $F1 = 2x\frac{Precision\ x\ Recall}{(Precision+Recall)}$
- **RougeL**
  - $RECALL = \frac{Length\ of\ LCS}{total\ words\ in\ reference\ text}$
  - $PRECISION = \frac{Length\ of\ LCS}{total\ words\ in\ generated\ text}$
  - $F1 = 2x\frac{Precision\ x\ Recall}{(Precision+Recall)}$

Just like ROUGE-1, both **ROUGE-2** and **ROUGE-L** use precision, recall, and F1 score to evaluate the quality of the generated text. The **F1 score** for each of these is the harmonic mean of precision and recall.

LLM Question Answering (QA)

- **Contextual Understanding**: LLMs are capable of understanding context to generate accurate answers based on a given passage or prompt.
- **Fine-Tuning for QA**: Pre-trained LLMs like BERT and GPT are often fine-tuned on specific question-answer datasets (e.g., SQuAD) to improve their performance in QA tasks.
- **Extractive QA**: The model extracts an answer directly from the input text (e.g., BERT).
- **Generative QA**: The model generates an answer based on the input, even when the exact answer isn't present in the text (e.g., GPT-3).
- **Metrics for QA**:
  - **Exact Match (EM)**: Measures how often the model's answer exactly matches the ground truth.
  - **F1 Score**: Measures overlap between predicted and true answers, focusing on precision and recall.

Zero Shot, One shot, n-shot

- **Zero-Shot Learning**:
  - The model performs a task without any specific examples during training.
  - LLMs use their vast knowledge base to generalize across tasks (e.g., answering a question on a topic it hasn't explicitly seen).
- **One-Shot Learning**:
  - The model learns to perform a task from just one training example.
  - LLMs can adjust based on a single input example (e.g., generating text in a specific style after being shown one example).
- **N-Shot Learning (few Shot)**:
  - The model learns from a smaller number of examples (n > 1) to perform a task.
  - LLMs leverage a small set of examples to understand patterns and provide more accurate outputs.

Multi-model LLMs

- **Definition**: Multi-modal LLMs process and generate outputs across different data types (e.g., text, images, audio, video).
- **Key Components**:
  - **Text and Vision Integration**: Combines language understanding with image interpretation (e.g., models like CLIP, DALL-E).
  - **Text and Audio**: Models capable of processing both text and spoken language (e.g., Speech-to-Text, Text-to-Speech systems).
- Examples
  - **CLIP**: Aligns images and text, enabling tasks like image captioning and visual search.
  - **DALL·E**: Generates images based on textual descriptions.
  - **Flamingo**: Combines vision and language for tasks like visual question answering.

Hallucination

- Large language models like GPT or T5 are trained to predict the next word in a sequence based on the statistical patterns of language in massive datasets.
- However, the model doesn't have **real-world knowledge** or **common sense**—it only has **learned statistical patterns** between words and sequences.

$$L = -\sum_{t=1}^{T} \log P(x_t | x_1, x_2, ..., x_{t-1})$$

- During training, models are optimized to reduce the **cross-entropy loss**

- The model becomes very good at predicting tokens that are **statistically likely** in context but may still fail when it requires **factual correctness.**
- In brief, **Statistically Most likely != Factually Correct**